

02_Run_Extraction: Straight-Line Coastdown Interval Detection

Validation_Aero_Test

February 12, 2026

1 Overview

This document describes the current straight-line interval detection used to extract coastdown runs. The implementation lives in `aero_validation/segmentation.py` and is exposed through the CLI script `Code/02_Run_Extraction/export_drag_runs.py`.

The algorithm:

- proposes candidate run starts using speed peak detection,
- extends forward until a stopping condition (turning/braking/re-acceleration/stop),
- enforces quality gates (straightness, duration),
- writes an intervals CSV used by downstream steps.

2 Inputs

From the cleaned dataset CSV (typically `Outputs/01_Data_Preparation/Aero_Validation_Signals_cleaned`):

- **Time:** `t_s` [s]
- **Speed:** `GPS1_VEL_MAG_ms` [m s^{-1}]
- **Steering:** `ICU_SteeringAngle_deg` [deg]
- **Optional brakes:** `ICU_Brakpresf_bar`, `ICU_Brakpresr_bar` [bar]

3 Preprocessing

Let t be the sample times and v the speed signal.

- Estimate the median sample period Δt_{med} from $\Delta t = \text{diff}(t)$.
- Convert time windows (seconds) to sample windows by $n \approx \text{round}(\Delta t_{\text{window}} / \Delta t_{\text{med}})$.
- If smoothing is enabled (default): smooth v with a centered rolling mean using a window derived from `peak_window_s`.
- If brake signals exist: combine $b(t) = \max(b_f, b_r)$ and lightly smooth it with a centered rolling mean (fixed small window).

4 Candidate peak detection

Peak detection is implemented in `detect_drag_peaks(t, v, config)`. It combines two mechanisms and returns sorted unique indices:

1. **Local maxima:** indices where $v_s[i] \geq v_s[i-1]$ and $v_s[i] \geq v_s[i+1]$.
2. **Derivative sign change:** indices where the speed derivative changes from clearly positive to non-positive.

Prominence gate For each local maximum, compute a prominence proxy

$$p = v_s[i] - P_{60}(v_s \text{ in a local window around } i),$$

and require $v_s[i] \geq \text{min_peak_speed}$ and $p \geq \text{peak_prominence}$.

5 Forward extension and end conditions

Given a start index p (peak), the algorithm extends forward until the first end condition triggers. Let i be the current index and $j = i + \text{accel_window_s}$ (converted to samples).

End conditions (evaluated sequentially):

- **Stop:** $v_s[i] \leq \text{stop_speed_th}$.
- **Turning:** $|\theta[i]| > \text{steer_thresh}$ sustained for `turn\_window_s`.
- **Braking:** $b[i] > \text{brake_thresh}$ or an instantaneous brake “spike” exceeds `brake\_spike\_thresh`.
- **Re-acceleration:** if $v_s[j] - v_s[i] > \text{reaccel_eps}$.

The returned interval is half-open: (p, e) where e is the first index after the accepted end.

6 Quality gates

After computing (p, e) :

- **Straightness:** the fraction of samples in $[p, e)$ where $|\theta| \leq \text{steer_thresh}$ must be at least `straight_ratio`.
- **Duration:** the run duration $t[e-1] - t[p]$ must satisfy `min_dur_s` $\leq$ duration $\leq$ `max_dur_s`.
- **Non-overlap:** candidate peaks that would overlap a previously accepted interval are skipped.

7 CLI usage and outputs

Export intervals

Code/02_Run_Extraction/export_drag_runs.py loads the cleaned dataset, runs segmentation, writes an intervals CSV, and generates an overview plot.

Default output paths:

- Intervals CSV: `Outputs/02_Run_Extraction/csv/drag_intervals.csv`
- Overview plot: `Outputs/02_Run_Extraction/plots/velocity_overview_with_runs.png`



Figure 1: Overview of detected coastdown intervals overlaid on the speed trace (generated by `export_drag_runs.py`).

Intervals CSV schema (key fields):

- `run_id`: 1..N
- `start_idx`, `end_idx`: index range into the input dataset (end exclusive)
- `t0_s`, `t1_s`, `duration_s`
- `v_start_ms`, `v_end_ms`
- `source_basename`, `row_count`: metadata for mismatch detection

Optional debug log

If `-debug-out` is provided, the debug variant `segment_drag_runs_with_debug` writes a per-peak decision log (accepted/rejected + reasons).

8 Default thresholds (CLI defaults)

The defaults below are the ones exposed by `export_drag_runs.py`:

<code>steer_thresh</code>	10 deg
<code>straight_ratio</code>	0.75
<code>min_peak_speed</code>	15.0 m/s
<code>peak_window_s</code>	0.5 s
<code>peak_prominence</code>	0.05 m/s
<code>reaccel_eps</code>	0.2 m/s
<code>accel_window_s</code>	0.5 s
<code>brake_thresh</code>	0.05 bar
<code>brake_spike_thresh</code>	0.1 bar/s
<code>turn_window_s</code>	0.3 s
<code>min_dur_s</code>	1.5 s
<code>max_dur_s</code>	20.0 s
<code>stop_speed_th</code>	0.2 m/s

9 Per-run outputs

The script `Code/O2_Run_Extraction/generate_run_outputs.py` consumes `drag_intervals.csv` and writes:

- `Outputs/02_Run_Extraction/csv/run_{XXX}.csv`: sliced time series per interval
- `Outputs/02_Run_Extraction/plots/run_{XXX}_velocity_steering_brake.png`: QC plots per run

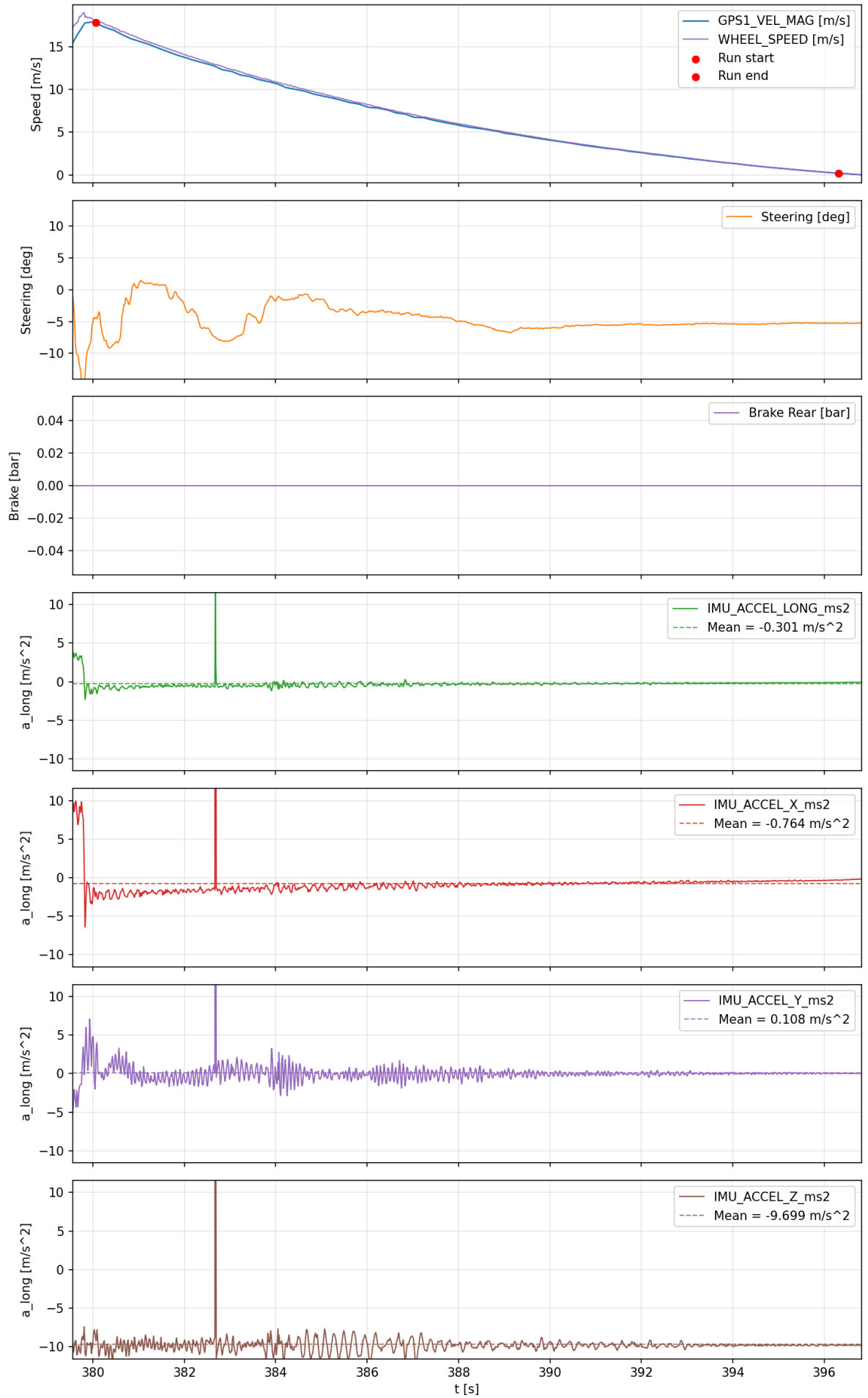


Figure 2: Example per-run QC plot (speed, steering, and brake traces) for Run 001.